Tapestreet v1.3: Mathematical Logic Review

Technical Specification for Big Widget LLC

This document outlines the underlying mathematical framework of the Tapestreet v1.3 engine. It details the transformation logic, key cycling protocols, and operational constraints designed for industrial-grade data sovereignty.

1. The Transformation Cipher (XOR)

The fundamental core of the Tapestreet engine is a bitwise exclusive-OR (XOR) operation. This atomic transformation operates at the byte level. Given a plaintext byte P_i and a corresponding key byte K_j, the ciphertext byte C_i is generated as follows:

$$C_i = P_i \oplus K_j$$

A primary benefit of the XOR operation in cryptographic contexts is its involutory nature. Decryption utilizes the exact same mathematical path as encryption:

$$P_i = C_i \oplus K_j$$

2. Key Cycling Logic (The Modulo Operator)

To process data streams of arbitrary length L using a fixed-length key S, the engine implements a cyclic indexing mechanism. Given a key set $K = \{k_0, k_1, ..., k_{n-1}\}$, the specific key byte utilized for the i^{th} byte of data is determined by the index j:

$$j = i \bmod n$$

Where:

- i is the linear byte offset of the data stream.
- n is the total length (cardinality) of the key set.
- j is the specific index within the key string.

3. Operational Constraints & Security Integrity

For Big Widget LLC compliance, the following mathematical and logical constraints are strictly enforced within the transformation engine:

> - **Key Entropy Requirement:** The key length must satisfy $|K| \geq 7$.
> - **Transformation Invariant:** Let $T(d, k)$ be the transformation function. The system ensures $T(T(d, k), k) = d$.
> - **Domain Restriction:** The operation is strictly bounded to the 8-bit binary domain: $P_i \in \{0, \&dots;, 255\}$.

4. Vault Mapping Logic

The engine applies a directional mapping to determine the operational mode (Encryption vs. Decryption) based on the file's environmental context within the system hierarchy:

$$Operation(f) = \{ Encrypt\ if\ path(f) \notin Vault;\ Decrypt\ if\ path(f) \in Vault \}$$

This ensures that any data entering the designated Vault directory is transformed into the protected *.bw* state, while data retrieved and moved to the */Decrypted* subdirectory is mathematically restored to its original source state.